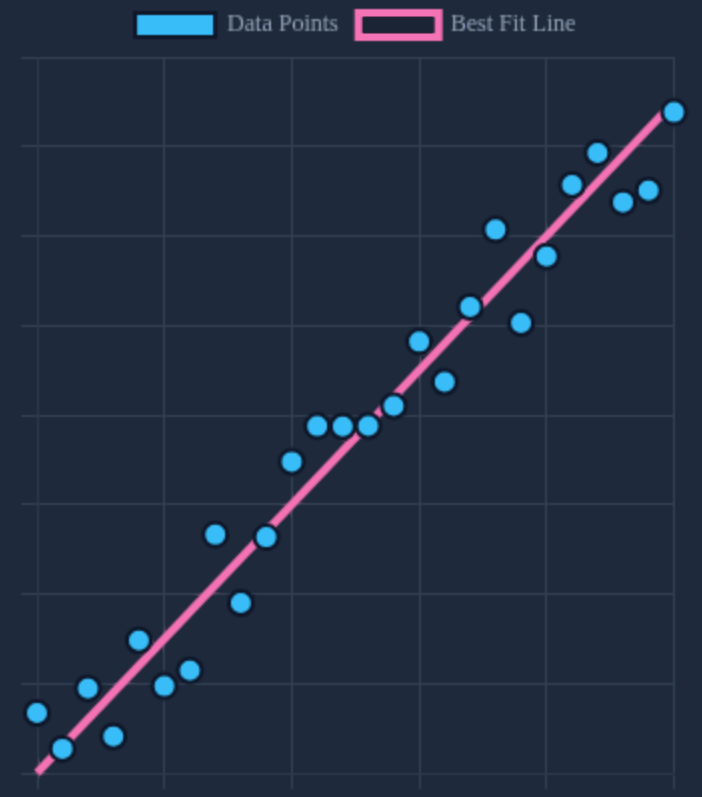
CS217 • WEEK 10

Linear Regression: From Intuition to Practice

> A Beginner's Guide to Predicting Numbers with Machine Learning



Model: Linear Regression (OLS)Status: Optimized

PRESENTER
ML Engineer

Agenda & Roadmap

Linear Regression: From Intuition to Practice

PHASE 1: INTUITION

01

What is Regression?

Problem definition & examples

02

Core Idea & Intuition

Fitting lines to data

03

The Math Behind It

Equation y = mx + c

PHASE 2: MECHANICS

04

Loss Functions

Mean Squared Error (MSE)

05

Optimization

Gradient Descent

06

Multiple Variables

Planes & Hyperplanes

07

Assumptions

Linearity, Homoscedasticity

PHASE 3: PRACTICE

08

Overfitting & Reg.

Ridge vs Lasso

09

Evaluation Metrics

R^2, RMSE, MAE

10

Practical Workflow

End-to-end steps

💡 Key Insight: We'll journey from **Intuition** to **Mathematical Mechanics**, and finally to **Practical Application**.

The Machine Learning Landscape

Where Linear Regression fits in the big picture

Machine Learning
Learning from data

Supervised Learning
Data has labels (Input → Output)

Regression — FOCUS TODAY
Predicts **continuous numbers**
- Ex: House Price ($500k)
- Ex: Temperature (24°C)

Classification
Predicts **categories/labels**
- Ex: Spam vs Not Spam
- Ex: Cat vs Dog

Unsupervised
No labels (Clustering, PCA)

Reinforcement
Learning via reward/punishment

Semi-Supervised

Key Takeaway: If the output is a number (e.g., price, height, score), use **Regression**. If it's a category (e.g., Yes/No, A/B/C), use Classification.

What Problem Does Regression Solve?

Predicting a continuous number from inputs

Rule of Thumb: Output = Number → Regression

DOMAIN	INPUT (FEATURES)		OUTPUT (TARGET)
	House Size sq ft, bedrooms, location	→	**$ Price** Continuous value
	Ad Spend TV, social media budget	→	**$ Sales Revenue** Projected income
	Study Hours Time spent, previous grades	→	**Exam Score** 0 - 100 scale
	Temperature Weather conditions	→	**Energy Usage** Kilowatt-hours (kWh)

Key Insight

If you are asking "How much?" or "How many?", you are likely solving a **Regression** problem.

From Decision Science to Regression

Understanding the optimization mindset behind machine learning

Define Problem

What variable do we want to optimize?

→

Objective Function

Mathematical formula for the goal

→

Minimize Error

Find parameters that reduce mistakes

→

Regression Model

The optimal solution found

TRADITIONAL OPTIMIZATION

In business operations, we often minimize costs:

```
Minimize Cost = Materials + Labor + Transport
```

MACHINE LEARNING (REGRESSION)

In regression, we minimize prediction mistakes:

```
Minimize Error = (Actual - Predicted)²
```

Key Insight: Machine Learning training is just **optimization in disguise**. We aren't "teaching" the computer; we are asking it to find parameters that minimize an error function.

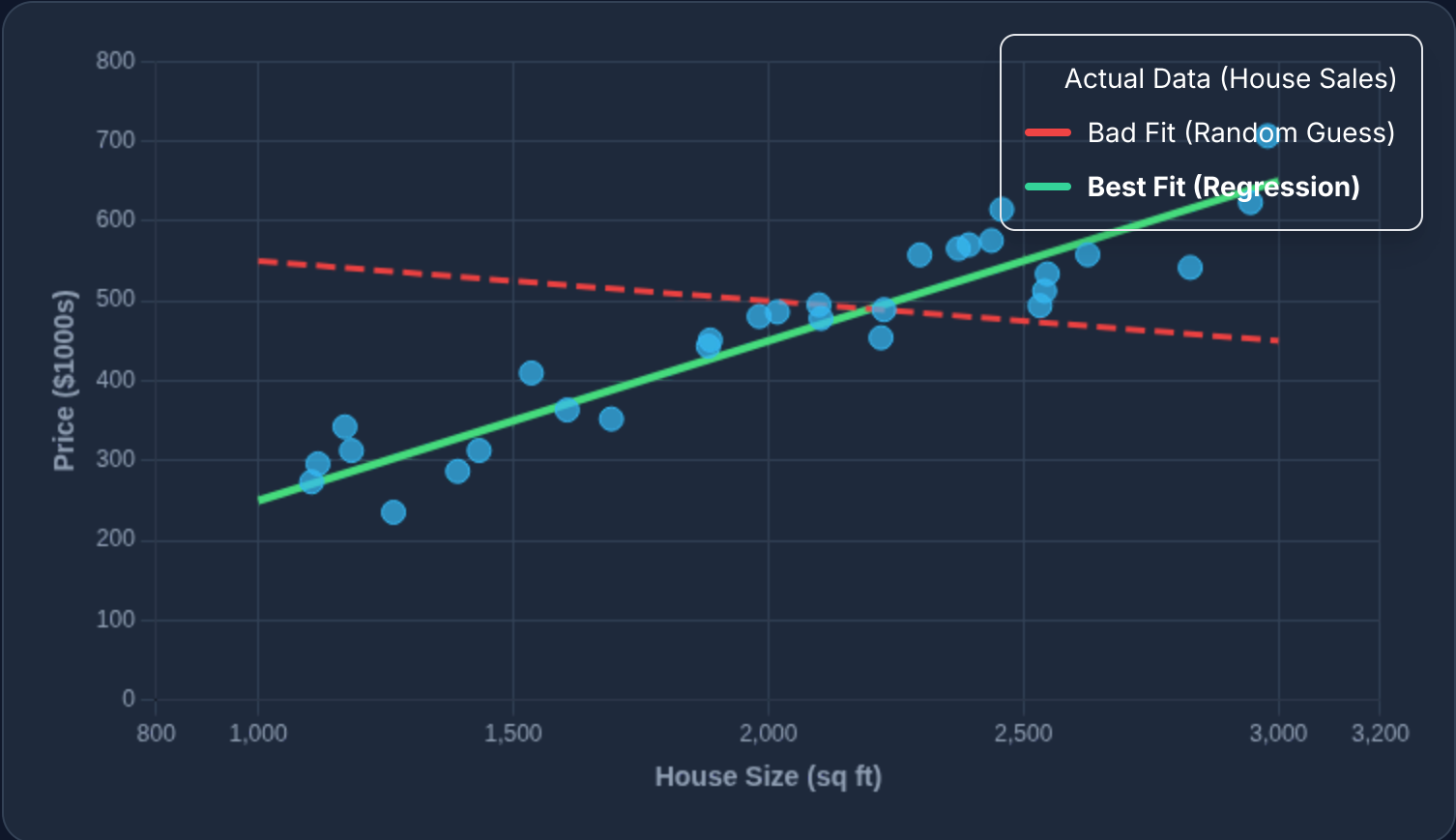
Core Idea: Fitting a Line

Finding the mathematical model that best describes reality

THE LINEAR MODEL

$$y = a + bx$$

- **y** — **Predicted Value** (Price)
- **x** — **Input Variable** (Size)
- **b** — **Slope** (Price per sq ft)
- **a** — **Intercept** (Base price)



> The goal of Linear Regression is simply to find the specific values of **a** and **b** that make the line

Key Insight: We don't draw the line by hand. We use math to calculate the exact **a** and **b** that minimize the total error distance.

Measuring Error (Residuals)

How wrong is our model for each data point?

SAMPLE DATA POINTS

`Error = Actual - Predicted`

SIZE (SQ FT)	ACTUAL PRICE (Y)	PREDICTED ($\hat{Y}$)	ERROR (RESIDUAL)
1000	52,000	51,000	+1,000
1200	61,000	63,000	-2,000
1400	68,000	67,000	+1,000

Simple Summation Problem

If we just add these errors up:

`1000 + (-2000) + 1000 = 0`

The total is zero, implying the model is perfect (which is false!).

Why Square the Errors?

Avoid Cancellation

Positive and negative errors cancel each other out.

```
Sum = (+1000) + (-2000) + (+1000) = 0
```

Make Everything Positive

Squaring ensures all mistakes count against the model.

```
Sum Sq = 1M + 4M + 1M = 6,000,000
```

Bonus: Squaring penalizes large errors heavily (e.g., an error of 10 becomes 100).

Key Insight: By squaring the errors, we turn individual positive and negative mistakes into a single **positive score** that represents total model performance.

Loss Function: MSE

Mean Squared Error — The score of how "wrong" the model is

THE OBJECTIVE FUNCTION

$$\text{Loss} = 1/n \; \Sigma(y - \hat{y})^2$$

y — **Actual Value**
The truth from data

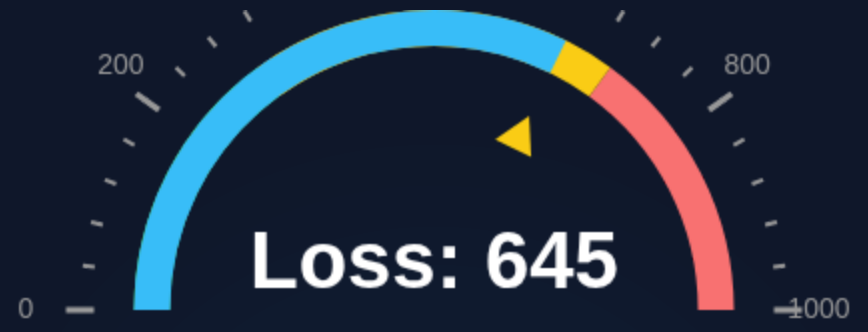
ŷ — **Predicted Value**
The model's guess (a + bx)

()² — **Squared Error**
Penalizes big mistakes

1/n — **Average**
Over all n data points

Model "Wrongness" Meter

We want this to be zero



Goal: Adjust a and b until the needle hits the green zone.

Key Insight: Training a regression model is simply **finding parameters that minimize the MSE**.

Optimization: Gradient Descent

How the model learns: Finding the bottom of the valley

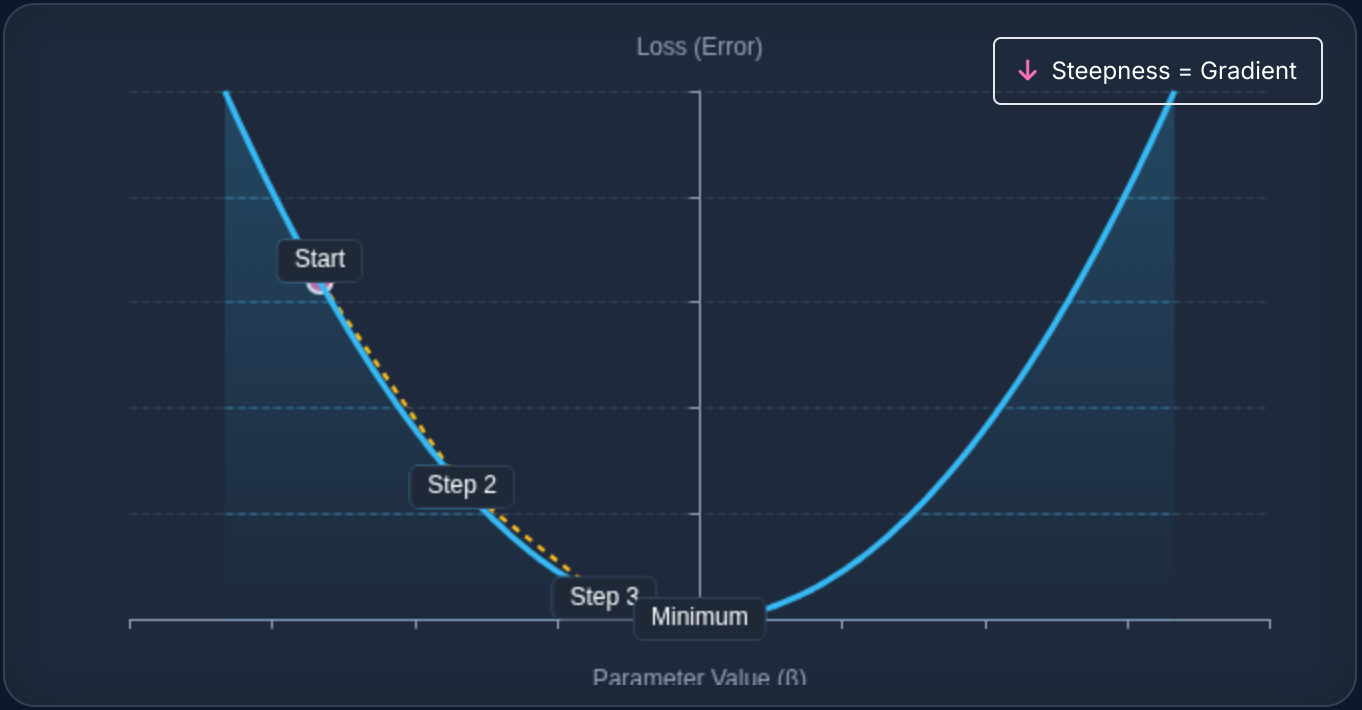
1. **Initialize Parameters**
   Start with random values for `a` and `b` (a random guess).

2. **Compute Loss**
   Calculate MSE to see how far off the guess is.

3. **Compute Gradient**
   Find the slope of the loss curve at the current point.

4. **Update Parameters**
   Move a small step downhill:
   `β_new = β_old - α × gradient`

5. **Repeat**
   Loop until the slope is near zero (convergence).



Learning Rate (α)
Controls step size

Too Big
Overshoots minimum

Too Small
Takes forever

Key Insight: Gradient descent is the engine of machine learning. It iteratively **adjusts parameters** to slide down the error mountain.

Two Ways to Solve

Comparing Iterative Optimization vs. Analytical Solution

Gradient Descent

Iteratively steps down the error hill.

$$\text{Repeat: } \beta := \beta - \alpha \nabla J(\beta)$$

Normal Equation

Solves for the minimum mathematically in one go.

$$\beta = (X^T X)^{-1} X^T y$$

FEATURE	GRADIENT DESCENT	NORMAL EQUATION
Dataset Size	Great for Large (n > 100k)	Good for Small (n < 10k)
Approach	Iterative (Many steps)	Analytical (One shot)
Computational Cost	$O(kn^2)$ Cheap per step	$O(n^3)$ Expensive matrix inversion

Key Insight: Use **Normal Equation** for small, quick datasets. Switch to **Gradient Descent** when data becomes too large for memory or matrix inversion.

Multiple Linear Regression

Adding more dimensions to capture complexity

1 VARIABLE

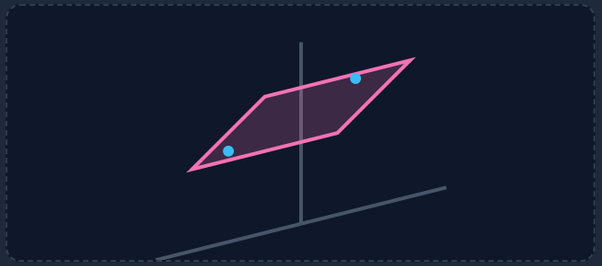
$$y = \beta_0 + \beta_1 x_1$$

A Line (2D)

With one input, we fit a straight line on a flat graph.

Visualizable on paper.

2 VARIABLES



$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

A Plane (3D)

With two inputs, we fit a flat sheet (plane) in 3D space.

Visualizable with 3D plots.

N VARIABLES

$$y = \beta_0 + \Sigma\ \beta_i x_i$$

A Hyperplane (nD)

With n inputs, we fit a geometric shape in n+1 dimensions.

Impossible to visualize, but math is same.

REAL WORLD EXAMPLE: HOUSE PRICES

EQUATION $\text{Price} = \text{Base} + \beta_1 \cdot \text{Size} + \beta_2 \cdot \text{Bedrooms} + \beta_3 \cdot \text{Location}$

Key Insight: The math doesn't change when we add variables. We are still finding the **"flattest" surface** that minimizes prediction error.

5 Key Assumptions

For Linear Regression to work reliably, these conditions should be met.

Linearity

The relationship between input (X) and output (Y) must be linear. If data curves, a straight line won't fit well.

Independence

Rows of data should not influence each other. Past values shouldn't predict future errors (no autocorrelation).

Homoscedasticity

Constant variance of errors. The "spread" of noise should stay the same across all predicted values.

No Multicollinearity

Input features shouldn't be highly correlated with each other. Each feature should provide unique info.

Normal Errors

Residuals (errors) should follow a normal distribution (bell curve). Important for calculating confidence intervals.

Key Insight: Don't just run `.fit()`. **Check these assumptions** to ensure your model's predictions and inferences are trustworthy.

Checking Linearity (Visual First!)

Before trusting the model, look at the data patterns.

Linear Pattern

✓ OK

Use Regression ✅

Curved (Quadratic)

✕ STOP

Need Polynomial ⚠️

Logarithmic

FIX

Transform: log(x) 🛠️

No Relationship

SKIP

Model won't help ❌

THE 3-STEP DIAGNOSTIC CHECK

1. **Scatter Plot**
   Visualize raw X vs Y first.
2. **Fit & Plot Residuals**
   Calculate error = y - ŷ.
3. **Check Randomness**
   Residuals should look like static noise.

Key Insight: Don't fly blind. **Plots reveal** if linear regression is appropriate or if you need a more complex model.

Heteroscedasticity

When the "spread" of errors changes across the data (The Funnel Effect).

Homoscedasticity (Good)

Constant Variance

Heteroscedasticity (Bad)

Variable Variance

How to Detect

- **Residual vs Fitted Plot:** Look for a cone or funnel shape opening left or right.
- **Breusch-Pagan Test:** Statistical test. P-value < 0.05 implies heteroscedasticity.
- **Scale-Location Plot:** Check if the trend line is sloping (should be horizontal).

How to Fix

- **Log Transformation:** Apply `log(y)`. Compresses large values, stabilizing variance.
- **Weighted Least Squares (WLS):** Give less weight to noisy observations.
- **Robust Standard Errors:** Adjust calculation of SE (e.g., HC3) to trust p-values.

Real World Example

Income vs. Education

→ **Low Education:** Almost everyone has low income. (Low Variance)

→ **High Education:** Some earn millions, some earn average. (High Variance)

Critical Insight: Linear regression still gives correct predictions (unbiased coefficients), but **confidence intervals and p-values become unreliable** if variance isn't constant.

Underfit vs. Good Fit vs. Overfit

Finding the "Goldilocks" zone in machine learning.

Underfitting

Model is too simple

Good Fit

Captures the signal

Overfitting

Memorizes the noise

Why Linear Regression Overfits?

- **Too Many Features:** More variables than data points (p > n).
- **Multicollinearity:** Features highly correlated inflate coefficients.
- **Noise Fitting:** Model tries to predict random errors.

How to Diagnose (The Signs)

Scenario	Training Error	Test Error	Verdict
Underfit	High	High	Too Rigid
Good Fit	Low	Low	Generalizes
Overfit	Very Low	High	Fails on New Data

Key Insight: A perfect score on training data is suspicious. We aim for a model that performs **equally well** on unseen test data.

Regularization: Ridge vs. Lasso

Preventing overfitting by penalizing complexity.

$$\text{Loss} = \underbrace{\Sigma(y - \hat{y})^2}_{\text{MSE (ERROR)}} + \underbrace{\lambda \cdot \text{Penalty}}_{\text{REGULARIZATION}}$$

$\lambda = 0 \rightarrow$ Standard OLS

$\lambda \uparrow \rightarrow$ Simpler Model

Ridge (L2)

$\lambda\Sigma\beta^2$

Penalty Type

Squared magnitude of coefficients. Penalizes large numbers heavily.

Effect

Shrinks coefficients towards zero, but **never exactly zero**.

Best Use Case

When you have **Multicollinearity** (correlated features) or want to keep all features.

Lasso (L1)

$\lambda\Sigma|\beta|$

Penalty Type

Absolute value of coefficients. Geometric "diamond" constraint.

Effect

Can shrink coefficients **exactly to zero**. Performs feature selection.

Best Use Case

When you have many **irrelevant features** and want a sparse model.

λ (Lambda) — Controls Strength: Higher λ = More Regularization = Less Overfitting

Key Insight: Regularization adds a "cost" to complexity. It forces the model to **fit the data** without overreacting to noise.

Evaluation Metrics

How do we know if our regression model is actually good?

MSE

STANDARD

$$\text{Loss} = (1/n)\ \Sigma(y - \hat{y})^2$$

Mean Squared Error. The average of squared differences. Penalizes large errors heavily, but the unit is "squared" (hard to interpret).

RMSE

BEST INTUITION

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Root Mean Squared Error. Puts the error back into original units (e.g., dollars). If RMSE = 5000, your prediction is off by ~$5k on average.

MAE

ROBUST

$$\text{MAE} = (1/n)\ \Sigma|y - \hat{y}|$$

Mean Absolute Error. The average absolute difference. Treats all errors equally. Use this if you have outliers you don't want to over-penalize.

R^2 Score

VARIANCE

$$R^2 = 1 - (\text{Error} / \text{Variance})$$

Coefficient of Determination. Represents the proportion of variance in the output that is explained by the input variables.

R^2 Score Guide

1.0	Perfect Fit (Suspicious)
0.8	Good / Strong Fit
0.5	Moderate Fit
0.0	Same as guessing mean
< 0	Worse than baseline

Pro Tip: RMSE is best for explaining accuracy to business stakeholders ("We are off by $X"). R^2 is best for comparing model power.

When to Use vs. Not Use

Linear regression is powerful, but it's not a magic hammer for every nail.

USE REGRESSION WHEN...

- **Output is Continuous:** Target variable is a number (e.g., price, height, time).
- **Linear Relationship:** Scatter plot shows a roughly straight line trend.
- **Random Residuals:** Error plot looks like static noise, no patterns.
- **Interpretability Matters:** You need to explain *why* the prediction was made (Coefficients).
- **Small-Medium Data:** Works great even with limited data points.

PERFECT EXAMPLES

House Price Prediction • Sales Demand Forecasting • Credit Risk Scoring

VS

DO NOT USE WHEN...

- **Output is Categorical:** Target is a class (Yes/No, Spam/Ham). Use Classification.
- **Strong Curvature:** Data follows a U-shape, S-shape, or circle.
- **Complex Interactions:** x1 effect depends on x2 heavily.
- **Exponential Growth:** Viral growth or saturation (S-curves).
- **High-Dim Noise:** More features than data points (p >> n) without regularization.

BETTER ALTERNATIVES

Polynomial Regression • Random Forest • Gradient Boosting • Neural Nets

Golden Rule: Don't force a straight line onto a curved world. **Match the tool** to the data pattern and your business objective.

Practical Workflow

A disciplined path from raw data to reliable predictions.

01

Define Problem

- Is output continuous?
- What is the business goal?
- Identify target variable (y)

02

Collect & Clean

- Handle missing values
- Remove outliers
- Scale features (StandardScaler)

03

Explore Data

- Scatter plots vs target
- Correlation matrix
- Check distributions

04

Check Linearity

- Visual Scatter Check
- Initial Residual Plot
- Transform Log(x) if curved

05

Build Model

- Baseline: Linear Regression
- Complex: Ridge / Lasso
- Poly features if needed

06

Evaluate

- Train / Test Split
- Calculate RMSE, MAE
- Check R^2 Score

07

Diagnostics

- Homoscedasticity check
- Normality of errors
- Multicollinearity (VIF)

08

Tune & Optimize

- Find best Lambda (α)
- Cross-Validation (CV)
- Feature Selection

09

Deploy

- Save model (pickle)
- API Integration
- Monitor Data Drift

Pro Tip: 80% of the work happens in steps 2, 3, and 4. If you feed garbage data into a perfect model, you still get garbage predictions.

Key Takeaways

Summary of Linear Regression fundamentals.

01

Continuous Prediction

Regression is used to predict numeric values (price, sales) using a linear equation `y = a + bx`.

02

Minimize MSE

The "best" line is found by minimizing the Mean Squared Error (MSE) between actual and predicted points.

03

Hyperplanes

With multiple variables, the "line" becomes a plane (3D) or hyperplane (nD). The math remains the same.

04

Diagnostic Plots

Always check scatter plots for linearity and residual plots for randomness. Patterns = problems.

05

Overfitting Risk

Complex models memorize noise. If Training Error is low but Test Error is high, you are overfitting.

06

Ridge vs. Lasso

Ridge shrinks coefficients (handles correlation). **Lasso** removes features (selects best inputs).

07

RMSE & R^2

RMSE tells you the average error in real units. **R^2** tells you the % of variance explained.

08

ML Foundation

Linear regression is the simplest building block. It is interpretable, fast, and the baseline for all ML.

Final Thought: Linear regression is **simple, interpretable, and effective**. Always try it first before moving to complex "black box" models.

Questions?

"Linear Regression is simply finding the best **hyperplane** that minimizes **prediction error**."